15.ROS2Launch 多节点启动与配置脚本

1.Launch 简介

到目前为止，每当我们运行一个 ROS 节点，都需要打开一个新的终端运行一个命令。机器人系统中节点很多，每次都这样启动好麻烦。有没有一种方式可以一次性启动所有节点呢？答案当然是肯定的，那就是 Launch 启动文件，它是 ROS 系统中多节点启动与配置的一种脚本。

ROS2 中，Launch 用于多节点启动和配置程序运行参数等功能，ROS2 的 Launch 文件格式有 xml、yaml 和 Python 格式。本节课程以 Python 格式的 Launch 文件为例，相对于另外两种格式，Python 格式的更加灵活：

- Python 拥有众多的函数库，可以在启动文件中使用；
- ROS2 通用启动特性和特定启动特性是用 Python 编写的，因此可以访问 XML 和 YAML 可能没有公开的启动特性。

使用 Python 语言编写 ROS2 launch 文件，最主要的是每个节点、文件、脚本等抽象成一个 action，用统一的接口来启动，主要结构是：

```
def generate_launch_description():
    return LaunchDescription([
        action_1,
        action_2,
        ...
        action_n
    ])
```

2.单个节点 Launch 文件

2.1 single_node.launch.py 创建

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

2） 输入“**ros2 pkg create launch_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**launch_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create launch_demo --build-type ament_pyth
on --dependencies rclpy
going to create a new package
```

3） 输入“**cd launch_demo/**”指令，切换到 launch_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd launch_demo/
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$
```

4） 输入“**mkdir launch**”指令，创建 launch 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$ mkdir launch
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$
```

5） 输入“**cd launch**”指令，切换到 launch 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$ cd launch
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$
```

6） 输入指令“**gedit single_node.launch.py**”编辑程序，，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@raspberrypi:~/hiwonder_ws/src/topic_demo/topic_demo$ vim topic_pub.py
```

```
from launch import LaunchDescription          # launch 文件的描述类

from launch_ros.actions import Node           # 节点启动的描述类
```

```
def generate_launch_description():                # 自动生成 launch 文件的函数
    return LaunchDescription([                    # 返回 launch 文件的描述信息
        Node(                                     # 配置一个节点的启动
            package='hello_world_demo',           # 节点所在的功能包
            executable='hello_world',             # 节点的可执行文件
        ),
    ])
```

7） 输入指令“**chmod +x single_node.launch.py**”，并按下回车，为保存的 single_node.launch.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ chmod +x single_node.launch.py
```

2.2 setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 **single_node.launch.py** 的程序入口写到 setup.py 文件中。

1） 输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$
```

2） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$ gedit setup.py
```

复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
from setuptools import find_packages, setup

import os
```

```
from glob import glob

(os.path.join('share', package_name, 'launch'), glob(os.path.join('launc
h', '*.launch.py'))),
```

```
from setuptools import find_packages, setup

import os
from glob import glob

package_name = 'launch_demo'

setup(
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share', package_name, 'launch'), glob(os.path.join('launch', '*.launch.py'))),
    ],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='ubuntu',
    maintainer_email='ubuntu@todo.todo',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
        ],
    },
)
```

2.3 编译运行

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 launch launch_demo single_node.launch.py**”，并按下回车，启动 single_node.launch.py 文件。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 launch launch_demo single_node.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-02-27-03-42-51-633328-ubuntu-12423
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [hello_world-1]: process started with pid [12425]
[hello_world-1] [INFO] [1709034172.203570493] [hello_world_demo]: Hello World
[hello_world-1] [INFO] [1709034172.706307904] [hello_world_demo]: Hello World
[hello_world-1] [INFO] [1709034173.208140756] [hello_world_demo]: Hello World
[hello_world-1] [INFO] [1709034173.710035370] [hello_world_demo]: Hello World
```

2.4 程序分析

通过 Node 描述一个 hello_world 节点的启动任务,包括节点所在的包名和执行文件,并返回 LaunchDescription 对象包含该节点任务,实现生成一个 launch 文件以描述和启动 hello_world 节点。

```
from launch import LaunchDescription                # launch文件的描述类
from launch_ros.actions import Node                 # 节点启动的描述类

def generate_launch_description():                  # 自动生成launch文件的函数
    return LaunchDescription([                      # 返回launch文件的描述信息
        Node(                                       # 配置一个节点的启动
            package='hello_world_demo',             # 节点所在的功能包
            executable='hello_world',               # 节点的可执行文件
        ),
    ])
```

3.多个节点 Launch 文件

3.1 multi_node.launch.py 创建

1） 新建一个命令行终端，然后输入指令：“**cd ros2_ws/src/launch_demo/launch**”指令，切换到 launch_demo 工作空间的 launch 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/launch_demo/launch
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$
```

2） 输入指令“**gedit multi_node.launch.py**”编辑程序复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ gedit multi_node.launch.py
```

```
from launch import LaunchDescription                # launch 文件的描述类

from launch_ros.actions import Node                 # 节点启动的描述类



def generate_launch_description():                  # 自动生成 launch 文件的函数

    return LaunchDescription([                      # 返回 launch 文件的描述信息
```

```
        Node(                                   # 配置一个节点的启动
            package='topic_demo',               # 节点所在的功能包
            executable='topic_custom_pub',              # 节点的可执行文件
        ),
        Node(                                   # 配置一个节点的启动
            package='topic_demo',               # 节点所在的功能包
            executable='topic_custom_sub',              # 节点的可执行文件名
        ),
    ])
```

3） 输入指令“**chmod +x multi_node.launch.py**”，并按下回车，为保存的 multi_node.launch.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ chmod +x multi_node.launch.py
```

3.2 编译运行

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 launch launch_demo multi_node.launch.py**”，并按下回车，

启动 multi_node.launch.py 文件。

3.3 程序分析

通过 Node 描述了两个 topic_demo 包下的节点启动任务(一个发布节点和一个订阅节点),并将两个节点任务加入到 LaunchDescription 对象中返回,生成一个描述式的 launch 文件自动启动这两个 topic 交互的节点任务。

```
from launch import LaunchDescription                # launch文件的描述类
from launch_ros.actions import Node                 # 节点启动的描述类

def generate_launch_description():                  # 自动生成launch文件的函数
    return LaunchDescription([                      # 返回launch文件的描述信息
        Node(                                       # 配置一个节点的启动
            package='topic_demo',                   # 节点所在的功能包
            executable='topic_pub',                 # 节点的可执行文件
        ),
        Node(                                       # 配置一个节点的启动
            package='topic_demo',                   # 节点所在的功能包
            executable='topic_sub',                 # 节点的可执行文件名
        ),
    ])
```

4.重映射 Launch 文件

ROS 社区中的资源非常多，当使用别人代码的时候，经常会发现通信的话题名称不太符合我们的要求，能否对类似的资源重新命名呢？为了提高软件的复用性，ROS 提供了资源重映射的机制。

4.1 remapping.launch.py 创建

1） 输入“**cd ros2_ws/src/launch_demo/launch**”指令，切换到 launch_demo 工作空间的 launch 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/launch_demo/launch
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$
```

2） 输入指令“**gedit remapping.launch.py**”编辑程序，复制下面程序。如需修改，再按下“**i**”即可修改。修改完成，按下“**Esc**”，输入“**：wq**”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ gedit remapping.launch.py
```

```
from launch import LaunchDescription            # launch 文件的描述类
from launch_ros.actions import Node             # 节点启动的描述类


def generate_launch_description():              # 自动生成 launch 文件的函数
    return LaunchDescription([                  # 返回 launch 文件的描述信息
        Node(                                   # 配置一个节点的启动
            package='topic_demo',               # 节点所在的功能包
            executable='topic_pub', # 节点的可执行文件
            remappings=[("/topic","topic_pub")]
        ),
    ])
```

3） 输入指令“**chmod +x remapping.launch.py**”，并按下回车，为保存的 remapping.launch.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ chmod +x remapping.launch.py
```

4.2 编译运行

1） 再新建一个命令行终端，输入“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 launch launch_demo remapping.launch.py**”，并按下回车，

启动 remapping.launch.py 文件。原先话题名称为“**/topic**”，重映射后话题名称为“**/topic_pub**”。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 launch launch_demo remapping.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-02-27-04-06-08-887548-ubuntu-14730
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [topic_custom_pub-1]: process started with pid [14732]
```

5） 然后再新建一个新的终端窗口，输入“**ros2 topic list**”指令，查看当前话题列表。原先话题名称为“**/topic**”，重映射后话题名称为“**/topic_pub**”。

```
ubuntu@ubuntu:~$ ros2 topic list
/parameter_events
/rosout
/topic_pub
ubuntu@ubuntu:~$
```

4.3 程序分析

通过 Node 动作描述了一个从 topic_demo 包下执行 topic_pub 节点的启动任务,并且通过 remappings 参数将节点发布的 topic 名称从"/topic"重映射为"topic_pub",最后将此节点任务添加到 LaunchDescription 对象并返回,目的就是自动生成一个描述文件的 launch 文件来启动描述中的节点任务,同时实现了话题名称的重映射配置。

```
from launch import LaunchDescription                    # launch文件的描述类
from launch_ros.actions import Node                     # 节点启动的描述类

def generate_launch_description():                      # 自动生成launch文件的函数
    return LaunchDescription([                          # 返回launch文件的描述信息
        Node(                                           # 配置一个节点的启动
            package='topic_demo',                   # 节点所在的功能包
            executable='topic_pub', # 节点的可执行文件
            remappings=[("/topic","topic_pub")]
        ),
    ])
```

5.设置参数 Launch 文件

5.1 param.launch.py 创建

1） 输入“**cd ros2_ws/src/launch_demo/launch**”指令，切换到 launch_demo 工作空间的 launch 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/launch_demo/launch
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$
```

2） 输入指令“**gedit param.launch.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ gedit param.launch.py
```

```
from launch import LaunchDescription                    # launch 文件的描述类
from launch.actions import DeclareLaunchArgument       # 声明 launch 文件内使用的 Argument 类
from launch.substitutions import LaunchConfiguration, TextSubstitution


from launch_ros.actions import Node                     # 节点启动的描述类



def generate_launch_description():                      # 自动生成 launch 文件的函数
   background_r_launch_arg = DeclareLaunchArgument(
      'background_r', default_value=TextSubstitution(text='0')     # 创建一个 Launch 文件内参数（arg）background_r
   )
   background_g_launch_arg = DeclareLaunchArgument(
      'background_g', default_value=TextSubstitution(text='120')    # 创建一个 Launch 文件内参数（arg）background_g
   )
   background_b_launch_arg = DeclareLaunchArgument(
```

```
    'background_b', default_value=TextSubstitution(text='90')   # 创建
一个 Launch 文件内参数（arg）background_b

  )



  return LaunchDescription([                                    # 返回 la
unch 文件的描述信息

    background_r_launch_arg,                                    # 调用以
上创建的参数（arg）

    background_g_launch_arg,

    background_b_launch_arg,

    Node(                                                       # 配置一个
节点的启动

        package='turtlesim',

        executable='turtlesim_node',                             # 节点所
在的功能包

        name='sim',                                             # 对节点重
新命名

        parameters=[{                                           # ROS 参数
列表

          'background_r': LaunchConfiguration('background_r'),   # 创建
参数 background_r

          'background_g': LaunchConfiguration('background_g'),   # 创建
参数 background_g

          'background_b': LaunchConfiguration('background_b'),   # 创建
```

```
参数 background_b

        }]

    ),

  ])
```

3） 输入指令“**chmod +x param.launch.py**”，并按下回车，为保存的 param.launch.py 赋予可执行权限。

5.2 编译运行

1） 新建命令行终端，然后输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

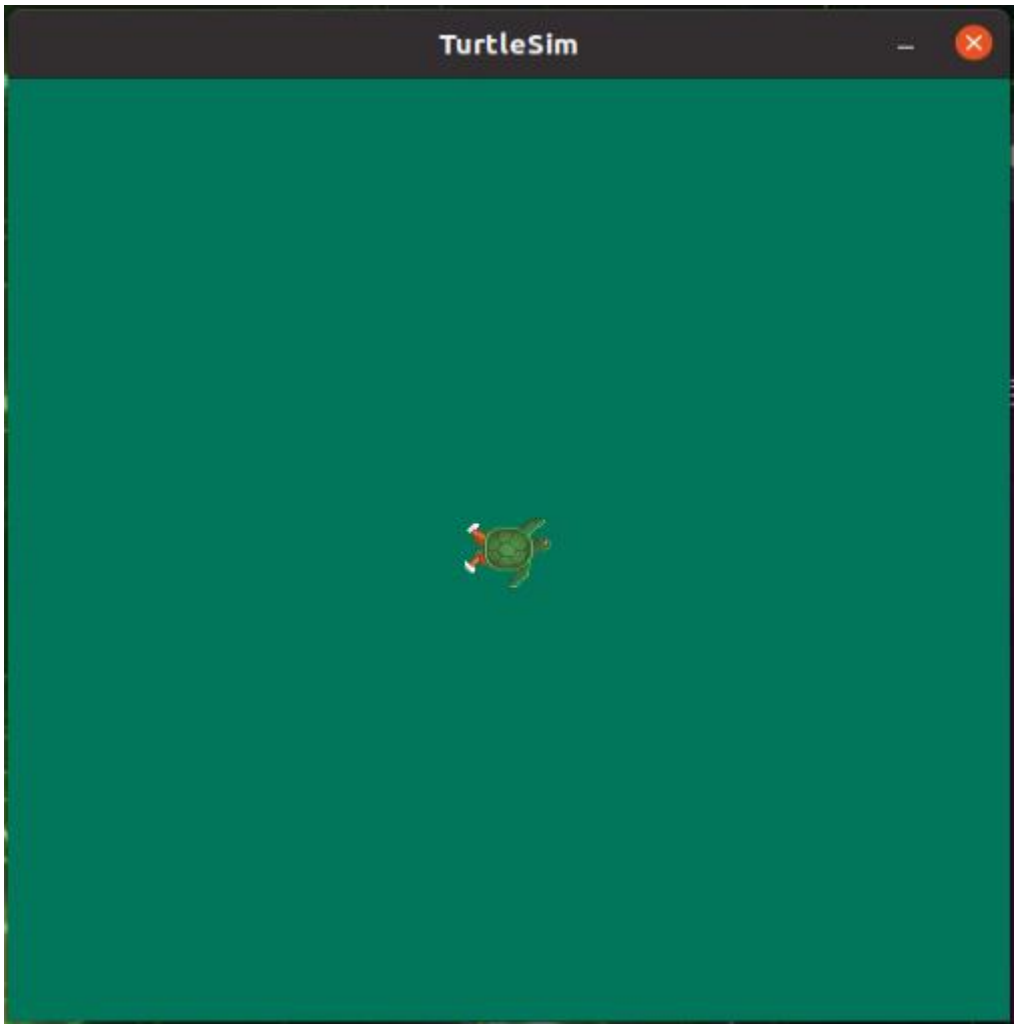
```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 launch launch_demo param.launch.py**”，并按下回车，启动 param.launch.py 文件。

```
ubuntu@ubuntu:~/ros2_ws$  ros2 launch launch_demo param.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-02
-27-04-16-29-029108-ubuntu-15736
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [turtlesim_node-1]: process started with pid [15739]
[turtlesim_node-1] [INFO] [1709036189.539386563] [sim]: Starting turtlesim with
 node name /sim
[turtlesim_node-1] [INFO] [1709036189.547147700] [sim]: Spawning turtle [turtle
1] at x=[5.544445], y=[5.544445], theta=[0.000000]
```



5.3 程序分析

通过 DeclareLaunchArgument 声明了 3 个参数 background_r、g、b,并在参数中设置默认值,然后通过 Node 描述 turtlesim 节点的启动任务,将这 3 个参数通过 LaunchConfiguration 注入到节点参数中,生成一个携带参数定义的 launch 文件,通过启动文件可以动态设置背景颜色参数以控制 turtle 模拟器的背景色。

```python
from launch import LaunchDescription                    # launch文件的描述类
from launch.actions import DeclareLaunchArgument       # 声明launch文件内使用的Argument类
from launch.substitutions import LaunchConfiguration, TextSubstitution

from launch_ros.actions import Node                     # 节点启动的描述类


def generate_launch_description():                      # 自动生成launch文件的函数
    background_r_launch_arg = DeclareLaunchArgument(
        'background_r', default_value=TextSubstitution(text='0')     # 创建一个Launch文件内参数（arg）background_r
    )
    background_g_launch_arg = DeclareLaunchArgument(
        'background_g', default_value=TextSubstitution(text='120')    # 创建一个Launch文件内参数（arg）background_g
    )
    background_b_launch_arg = DeclareLaunchArgument(
        'background_b', default_value=TextSubstitution(text='90')   # 创建一个Launch文件内参数（arg）background_b
    )

    return LaunchDescription([                                      # 返回launch文件的描述信息
        background_r_launch_arg,                                    # 调用以上创建的参数（arg）
        background_g_launch_arg,
        background_b_launch_arg,
        Node(                                                       # 配置一个节点的启动
            package='turtlesim',
            executable='turtlesim_node',                            # 节点所在的功能包
            name='sim',                                             # 对节点重新命名
            parameters=[{                                           # ROS参数列表
                'background_r': LaunchConfiguration('background_r'),    # 创建参数background_r
                'background_g': LaunchConfiguration('background_g'),    # 创建参数background_g
                'background_b': LaunchConfiguration('background_b'),    # 创建参数background_b
            }]
        ),
    ])
```

6. YAML 参数 Launch 文件

6.1 turtlesim.yaml 文件创建

1) 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/launch_demo/**”：

```
ubuntu@ubuntu:~$ cd ros2_ws/src/launch_demo/
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$
```

2) 然后输入指令“**mkdir config**”创建一个 config 文件夹：

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$ mkdir config
```

3) 输入指令“**cd config**”进入到 config 文件夹内：

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$ cd config/
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/config$
```

4) 接着输入指令“**gedit turtlesim.yaml**”创建 turtlesim.yaml 文件，并且将以下内容粘贴到 yaml 标签文件当中，然后使用快捷键“ctrl + s”保存退出即可。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/config$ gedit turtlesim.yaml
```

```
/turtlesim2/sim:

  ros__parameters:

    background_b: 0

    background_g: 0

    background_r: 0
```

6.2 param_yaml.launch.py 创建

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/launch_demo/launch**”指令，切换到 launch_demo 工作空间的 launch 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/launch_demo/launch
```

1） 输入指令“**gedit param_yaml.launch.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ gedit param_yaml.launch.py
```

```
import os



from ament_index_python.packages import get_package_share_directory  #
查询功能包路径的方法



from launch import LaunchDescription   # launch 文件的描述类

from launch_ros.actions import Node    # 节点启动的描述类
```

```
def generate_launch_description():     # 自动生成 launch 文件的函数
  config = os.path.join(               # 找到参数文件的完整路径
    get_package_share_directory('launch_demo'),
    'config',
    'turtlesim.yaml'
    )

  return LaunchDescription([           # 返回 launch 文件的描述信息
    Node(                              # 配置一个节点的启动
      package='turtlesim',             # 节点所在的功能包
      executable='turtlesim_node',  # 节点的可执行文件名
      namespace='turtlesim2',          # 节点所在的命名空间
      name='sim',                      # 对节点重新命名
      parameters=[config]              # 加载参数文件
    )
  ])
```

2） 新建一个命令行终端，输入指令“**chmod +x param_yaml.launch.py**”，并按下回车，为保存的 param_yaml.launch.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ chmod +x param_yaml.launch.py
```

6.3 setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将

param_yaml.launch.py 的程序入口写到 setup.py 文件中。

1） 输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$
```

2） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件。复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo$ gedit setup.py
```

```
(os.path.join('share', package_name, 'config'), glob(os.path.join('confi
g', '*.*'))),
```

```
12    data_files=[
13        ('share/ament_index/resource_index/packages',
14            ['resource/' + package_name]),
15        ('share/' + package_name, ['package.xml']),
16        (os.path.join('share', package_name, 'launch'), glob(os.path.join('launch', '*.launch.py'))),
17        (os.path.join('share', package_name, 'config'), glob(os.path.join('config', '*.*'))),
18    ],
19    install_requires=['setuptools'],
```

6.4 编译运行

1） 新建一个命令行终端，然后输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

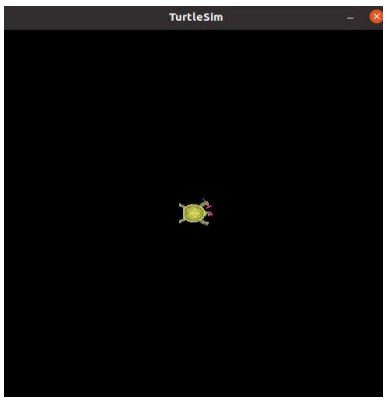
```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 launch launch_demo param_yaml.launch.py**”，并按下回车，启动 param_yaml.launch.py 文件。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 launch launch_demo param_yaml.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-02
-27-04-39-14-974052-ubuntu-17112
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [turtlesim_node-1]: process started with pid [17114]
[turtlesim_node-1] [INFO] [1709037555.428380310] [turtlesim2.sim]: Starting tur
tlesim with node name /turtlesim2/sim
[turtlesim_node-1] [INFO] [1709037555.437715469] [turtlesim2.sim]: Spawning tur
tle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
^C[WARNING] [launch]: user interrupted with ctrl-c (SIGINT)
[turtlesim_node-1] [INFO] [1709037575.107229022] [rclcpp]: signal_handler(signa
l_value=2)
```



6.5 程序分析

通过 Node 描述一个 turtlesim 节点的启动任务,设置了包名、执行文件名、命名空间和名称标签,并通过 parameters 从指定路径加载配置文件 turtlesim.yaml,最后将此任务添加到 LaunchDescription 中返回,生成一个用来描述和启动带参数的 turtlesim 节点的 launch 文件。

```python
import os

from ament_index_python.packages import get_package_share_directory  # 查询功能包路径的方法

from launch import LaunchDescription    # launch文件的描述类
from launch_ros.actions import Node     # 节点启动的描述类


def generate_launch_description():      # 自动生成launch文件的函数
    config = os.path.join(              # 找到参数文件的完整路径
        get_package_share_directory('launch_demo'),
        'config',
        'turtlesim.yaml'
        )

    return LaunchDescription([          # 返回launch文件的描述信息
        Node(                           # 配置一个节点的启动
            package='turtlesim',        # 节点所在的功能包
            executable='turtlesim_node',# 节点的可执行文件名
            namespace='turtlesim2',     # 节点所在的命名空间
            name='sim',                 # 对节点重新命名
            parameters=[config]         # 加载参数文件
        )
    ])
```

7.Launch 文件启动 Launch

7.1 launch_include.launch.py 创建

1） 输入“**cd ros2_ws/src/launch_demo/launch**”指令，切换到 launch_demo 工作空间的 launch 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/launch_demo/launch
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$
```

2） 输入指令“**gedit launch_include.launch.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ gedit launch_include.launch.py
```

```
import os


from ament_index_python.packages import get_package_share_directory  # 查询功能包路径的方法


from launch import LaunchDescription                # launch 文件的描述类
from launch.actions import IncludeLaunchDescription  # 节点启动的描述类
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import GroupAction              # launch 文件中的执行动作
from launch_ros.actions import PushRosNamespace      # ROS 命名空间配置


def generate_launch_description():                  # 自动生成 launch 文件
```

```
的函数
   hello_world = IncludeLaunchDescription(        # 包含指定路径下的另外一
个 launch 文件
      PythonLaunchDescriptionSource([os.path.join(
         get_package_share_directory('launch_demo'), 'launch'),
         '/single_node.launch.py'])
      )


   return LaunchDescription([                          # 返回 launch 文件的描述
信息
      hello_world
   ])
```

3） 输入指令“**chmod +x launch_include.launch.py**”，并按下回车，为保存的 launch_include.launch.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/launch_demo/launch$ chmod +x launch_include.launch.py
```

7.2 编译运行

1） 新建一个命令行终端，然后输入“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
ubuntu@ubuntu:~/ros2_ws$
```

4） 输入指令“**ros2 launch launch_demo launch_include.launch.py**”，并按下回车，启动 **launch_include.launch.py** 文件。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 launch launch_demo launch_include.launch.py
[INFO] [launch]: All log files can be found below /home/ubuntu/.ros/log/2024-02-27-04-43-20-087784-ubuntu-17614
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [hello_world-1]: process started with pid [17616]
[hello_world-1] [INFO] [1709037800.582412170] [hello_world_demo]: Hello World
[hello_world-1] [INFO] [1709037801.084214006] [hello_world_demo]: Hello World
[hello_world-1] [INFO] [1709037801.586014531] [hello_world_demo]: Hello World
```

7.3 程序分析

通过 IncludeLaunchDescription 动作从指定路径引入了另一个 launch 文件 single_node.launch.py 描述的任务,并将这个任务加入到 LaunchDescription 对象返回,生成一个主 launch 文件以包含和运行在子 launch 文件中定义的节点任务。

```python
import os

from ament_index_python.packages import get_package_share_directory  # 查询功能包路径的方法

from launch import LaunchDescription                # launch文件的描述类
from launch.actions import IncludeLaunchDescription  # 节点启动的描述类
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import GroupAction              # launch文件中的执行动作
from launch_ros.actions import PushRosNamespace     # ROS命名空间配置

def generate_launch_description():                  # 自动生成launch文件的函数
    hello_world = IncludeLaunchDescription(        # 包含指定路径下的另外一个launch文件
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('launch_demo'), 'launch'),
            '/single_node.launch.py'])
        )

    return LaunchDescription([                      # 返回launch文件的描述信息
        hello_world
    ])
```